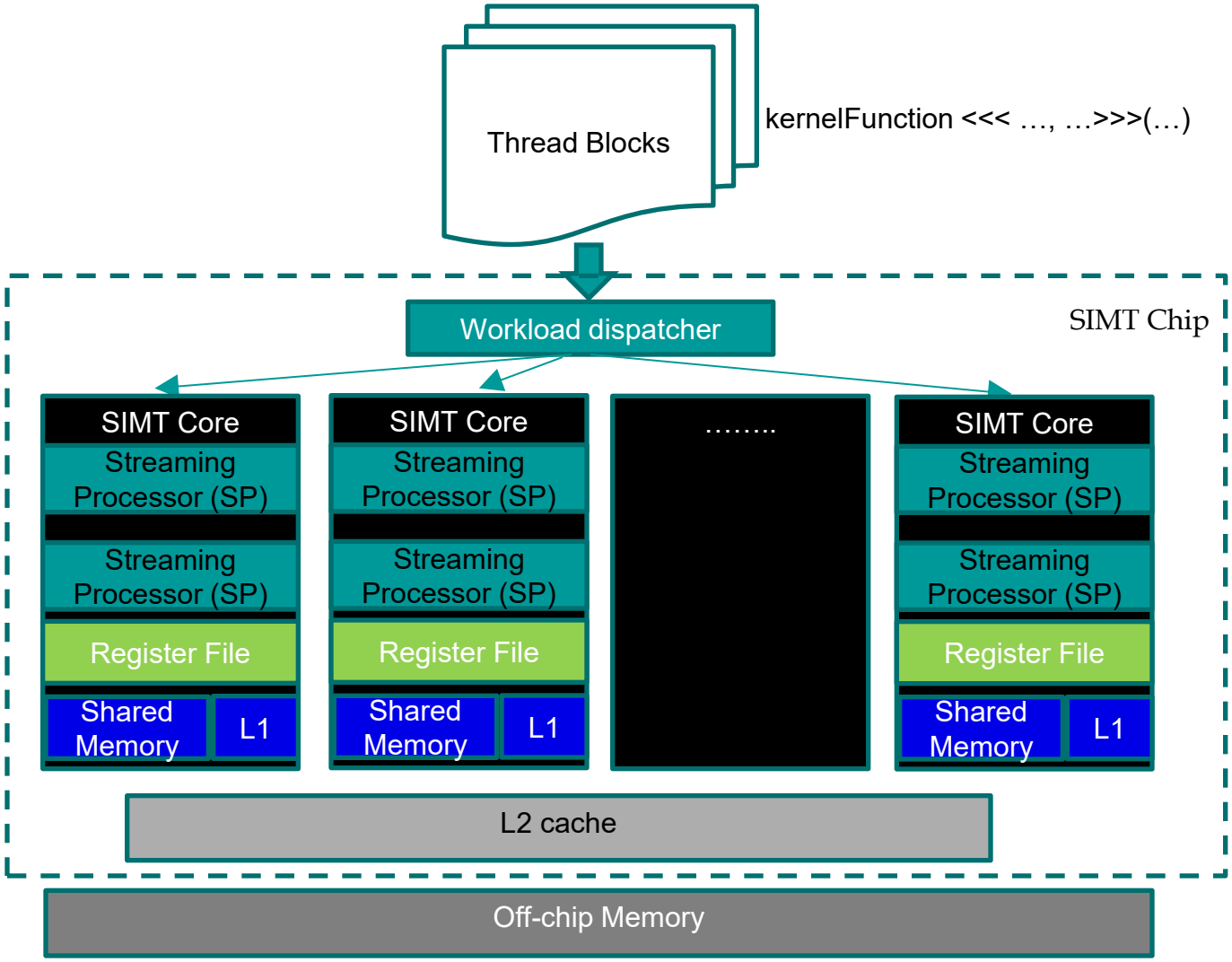
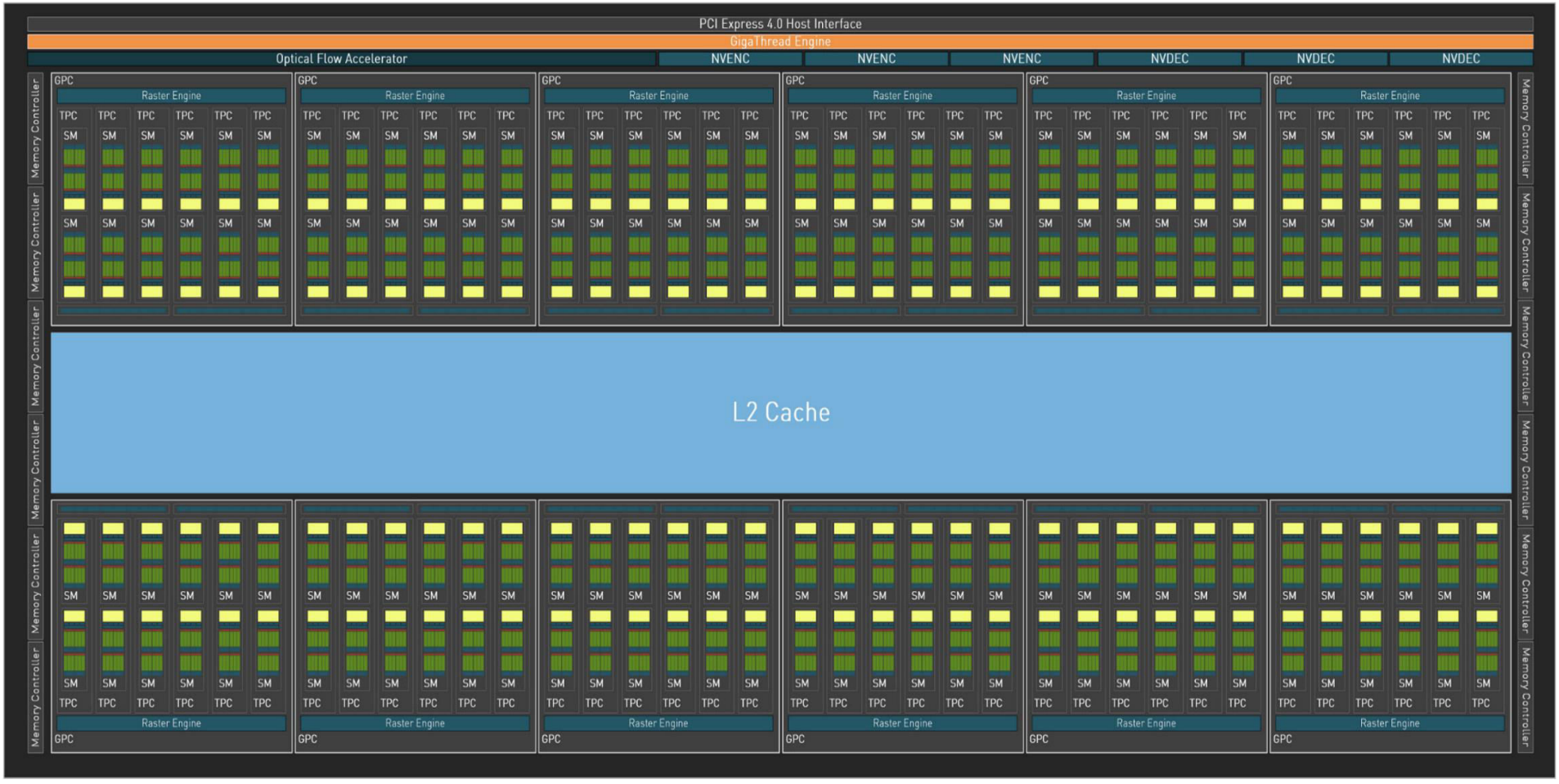
GPU Microarchitecture:
Hardware-Based Vectorization + Fine-Grain Multithreading

Huiyang Zhou

GPU Architecture



NVIDIA Ada-Lovelace GPU Architecture



NVIDIA Ada-Lovelace GPU SM Architecture

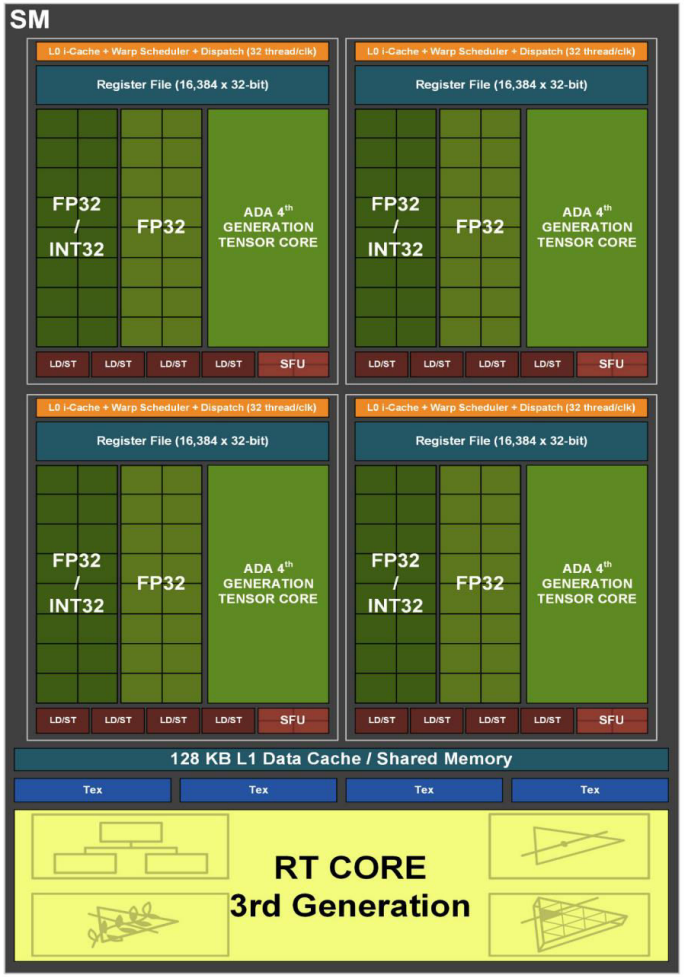
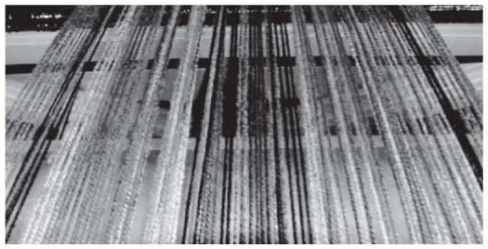


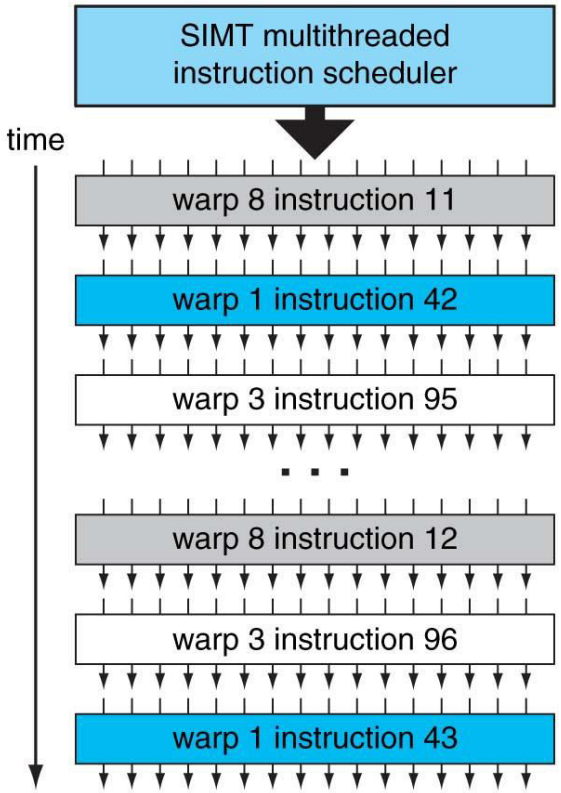
Figure 5. Ada Streaming Multiprocessor (SM)

Execution Model: Thread Hierarchy

- Kernels are launched in grids, which consists of many thread blocks.
- Thread blocks are assigned to SIMT cores / SMs
  - Threads in one thread block can communicate through shared memory
- Threads in a thread blocks are group to warps (32 threads per warp)
  - Warp is the basic scheduling unit
  - Threads in a warp proceed in the SIMD mode (i.e., single PC while each thread has its own register file)

Fine-Grain Multi-Threaded Execution in an SM





The principle to achieve high throughput:

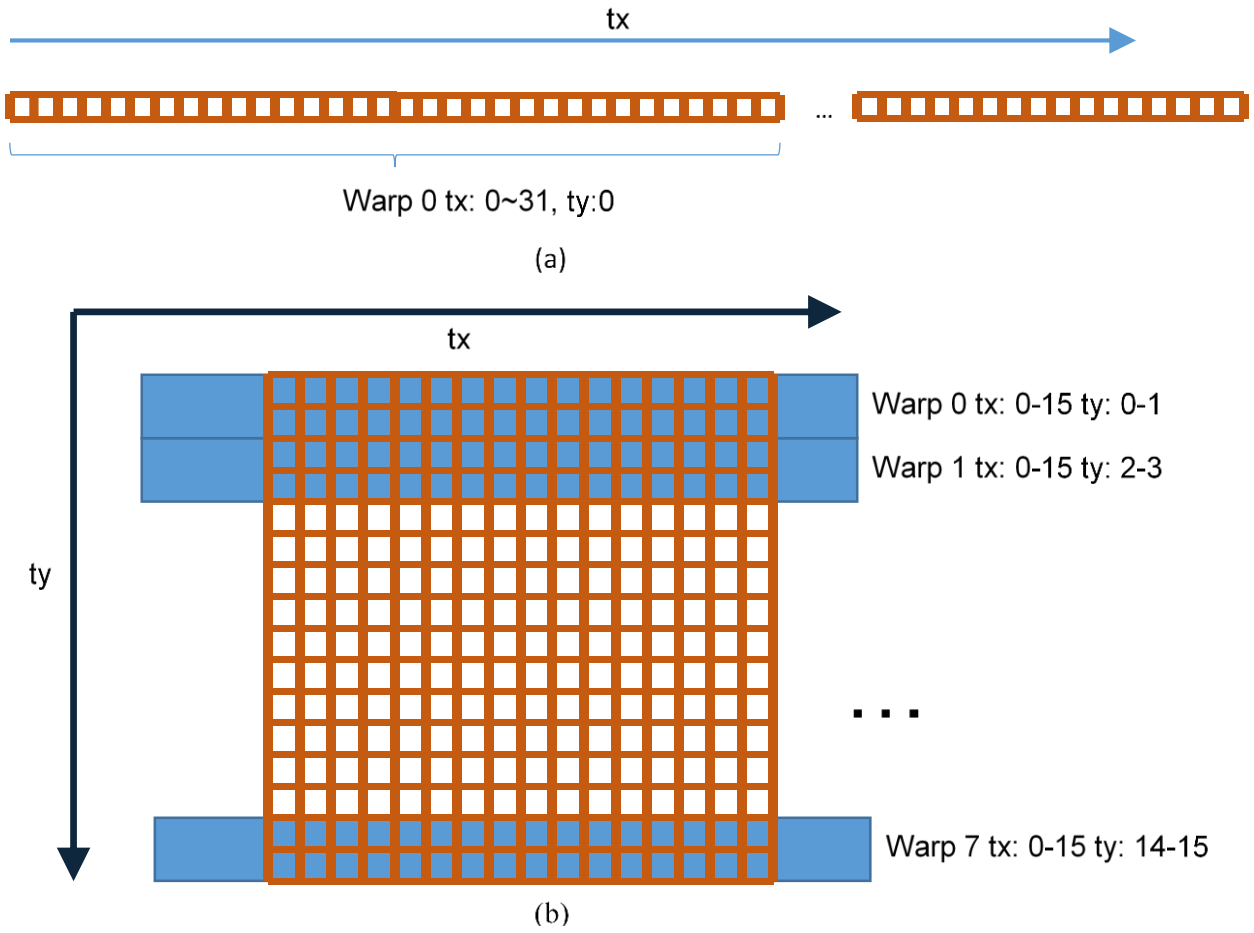
- Thread-level parallelism to hide instruction execution latency
- Effective for short latency instructions
- Off-chip memory accesses still a challenge

Hardware-based Vectorization

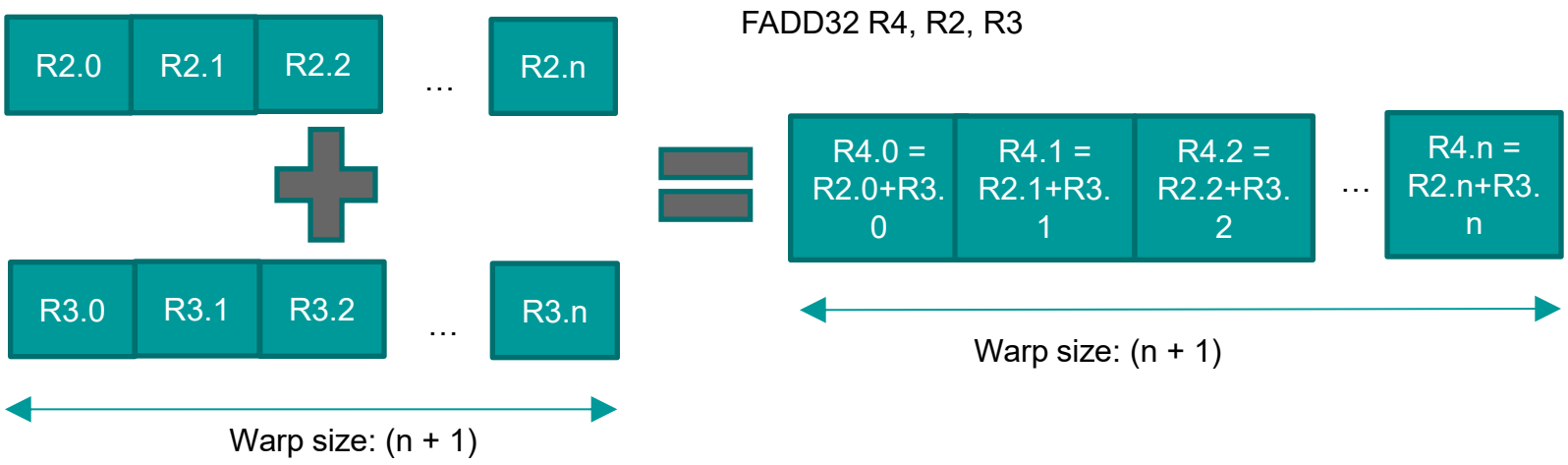
- Scalar ISA (vector-add kernel)

```
/*0000*/   /*0x1100e804        */          MOV32 R1, g [0x4];                  /*base */
/*0004*/   /*0x1100ea08        */          MOV32 R2, g [0x5];                  /*base */
/*0008*/   /*0x6004000500000003*/          IMAD32I.U16 R1, R0L, 0x4, R1;       /*index */
/*0010*/   /*0x6004000900000003*/          IMAD32I.U16 R2, R0L, 0x4, R2;       /*index */
/*0018*/   /*0xd00e020d80c00780*/          GLD.U32 R3, global14 [R1];          /*load*/
/*0020*/   /*0xd00e040980c00780*/          GLD.U32 R2, global14 [R2];          /*load*/
/*0028*/   /*0x1100ec04        */          MOV32 R1, g [0x6];                  /* base */
/*002c*/   /*0xb0020608        */          FADD32 R2, R3, R2;                  / * add */
/*0030*/   /*0x6004000500000003*/          IMAD32I.U16 R1, R0L, 0x4, R1;       /* index */
/*0038*/   /*0xd00e0209a0c00781*/          GST.U32 global14 [R1], R2;          /* store */
```

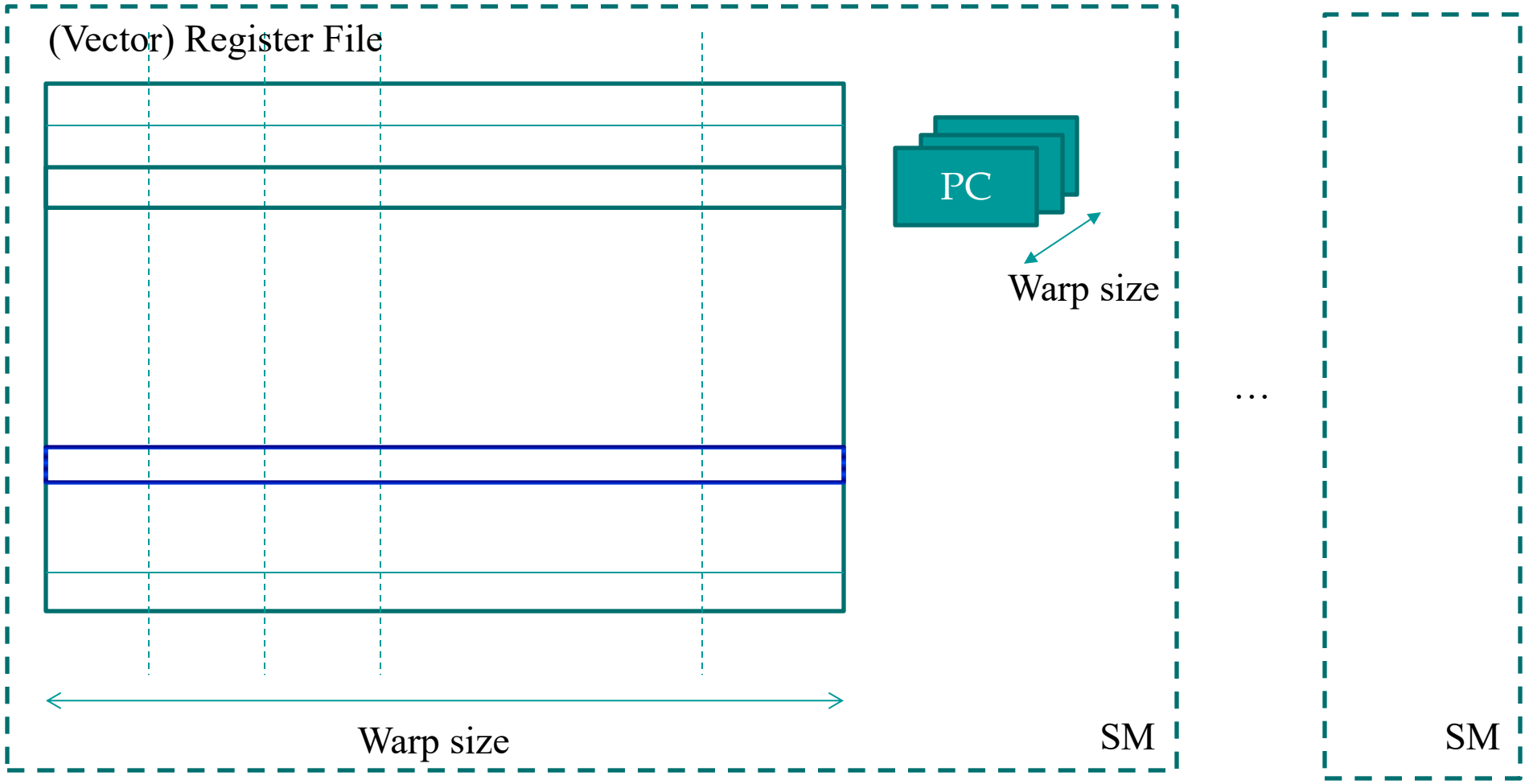
Hardware-based Vectorization: Warp Formation



SIMD execution using scalar instructions



GPU Architectural State



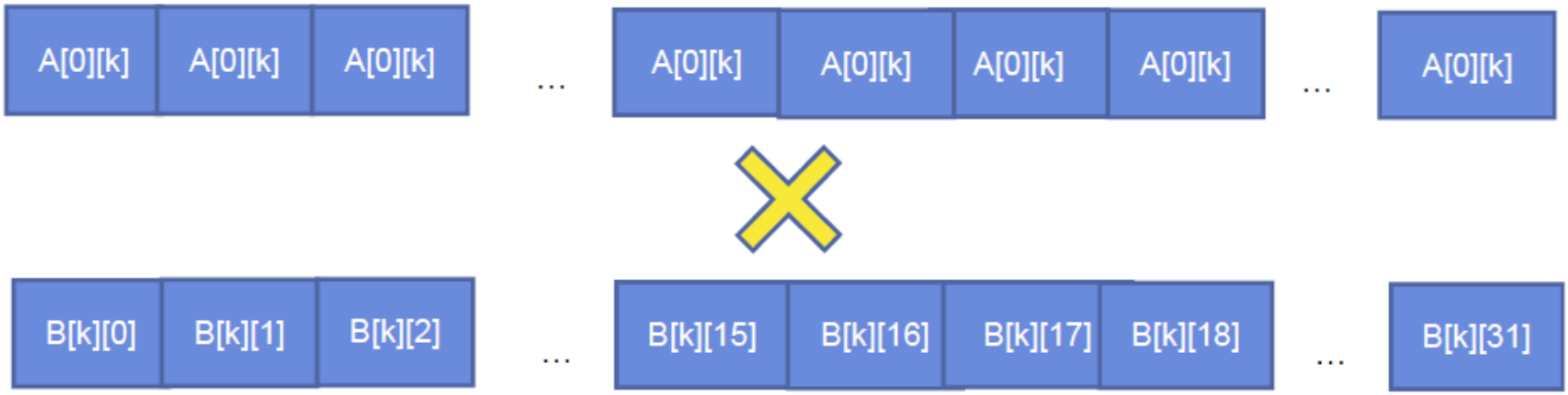
HW-based Vectorization on Matrix Multiplication

```
for(k = 0; k < N, k++)
        sum += A[idy][k] * B[k][idx];
C[idy][idx] = sum;
```

If a TB is configured as 32x8.

The vectorized operation for warp 0 becomes

```
for(k = 0; k < N, k++)
        C[0][0:31] += A[0][k] * B[k][0:31];
```



HW-based Vectorization on Matrix Multiplication (cont'd)

```
for(k = 0; k < N, k++)
        sum += A[idy][k] * B[k][idx];
C[idy][idx] = sum;
```

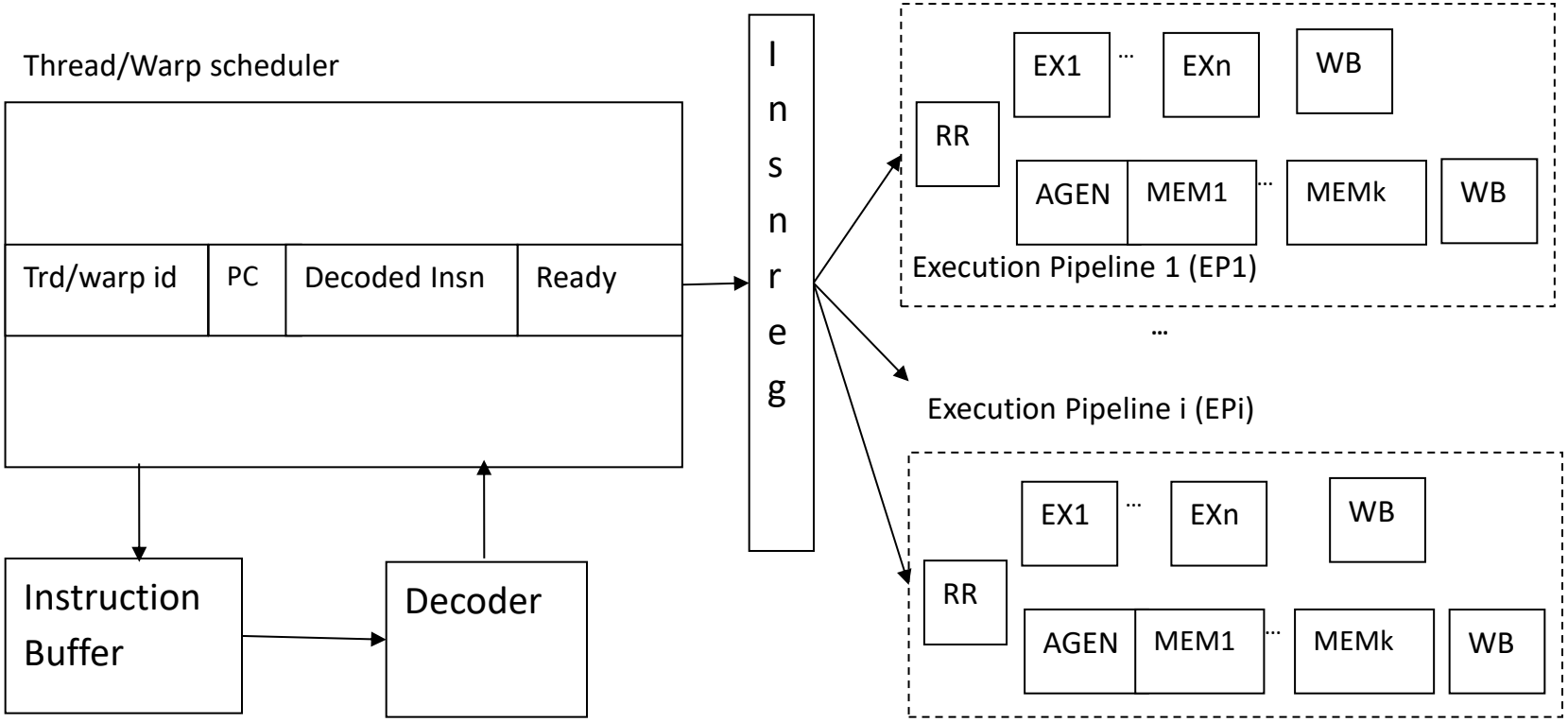
If a TB is configured as 16x16.

The vectorized operation for warp 0 becomes

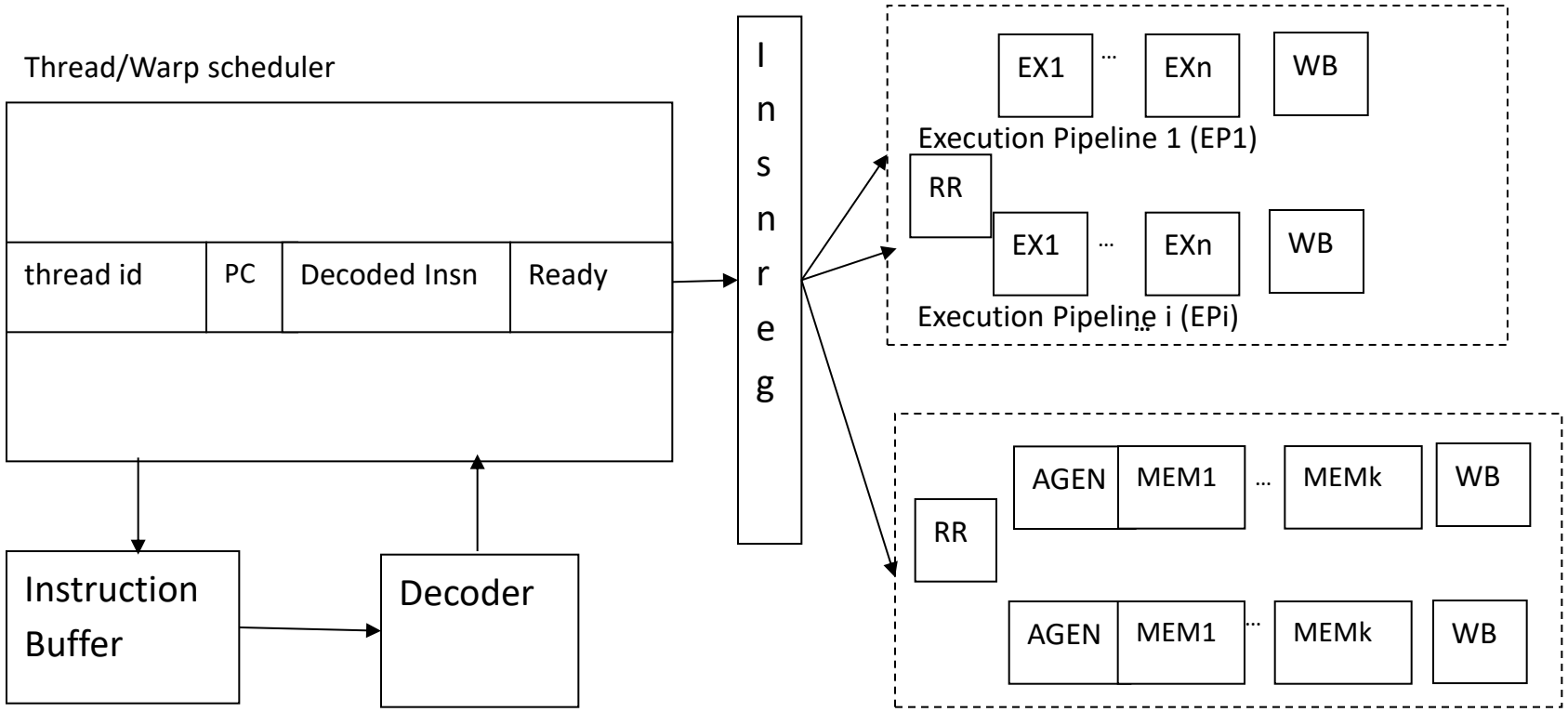
```
for(k = 0; k < N, k++){
        C[0][0:15] += A[0][k] * B[k][0:15];
        C[1][0:15] += A[1][k]* B[k][0:15];}
```



SM Microarchitecture



SM Microarchitecture

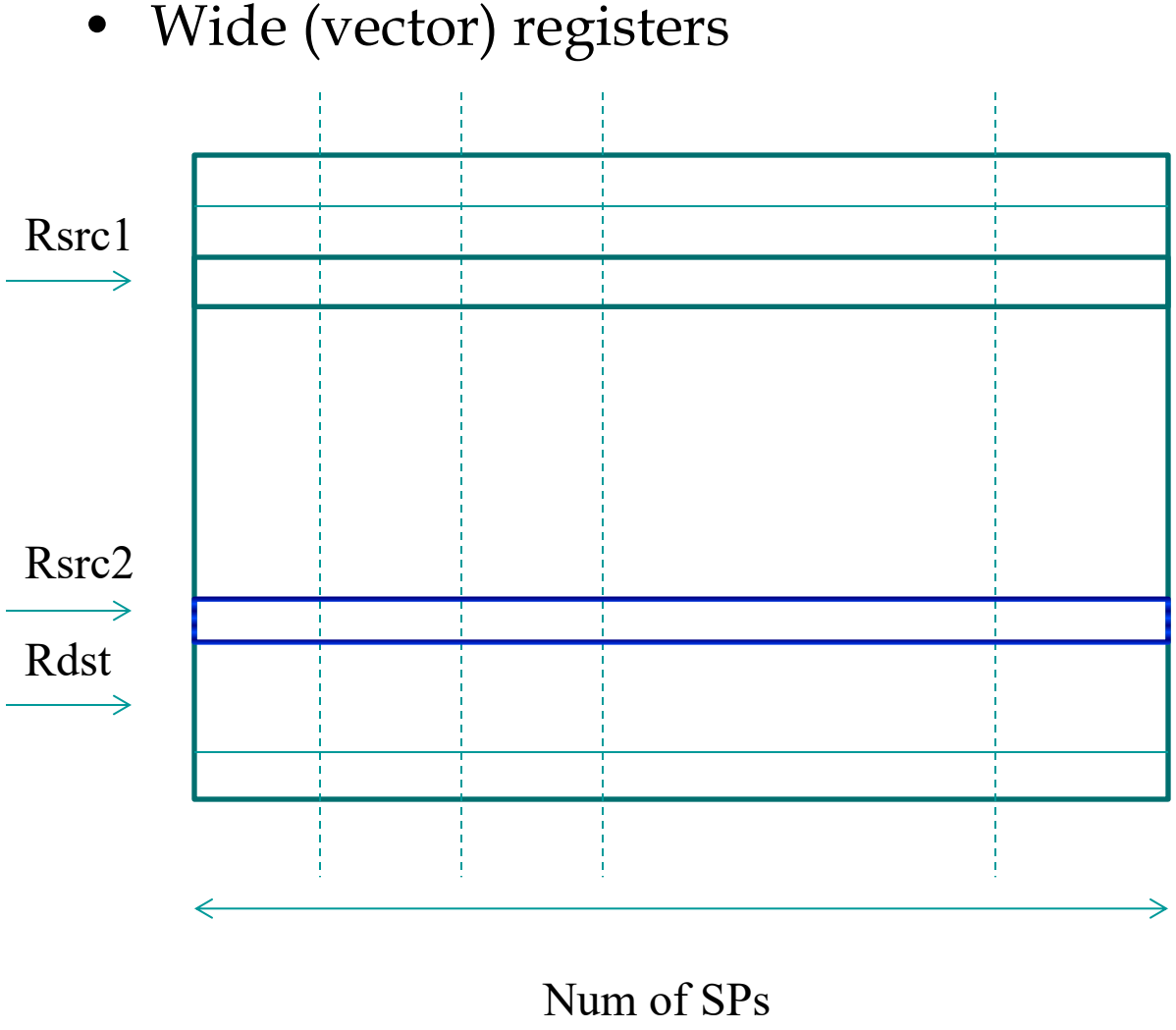


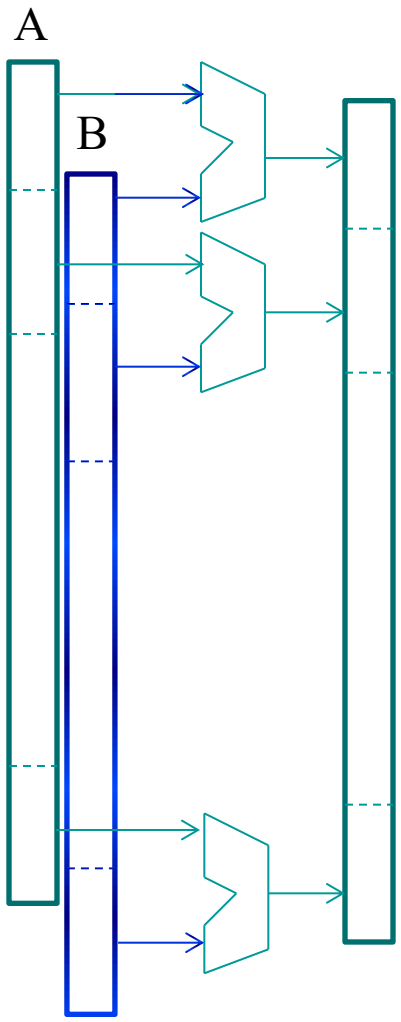
Pipeline Execution Diagram

cycle	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	
f1 = f2 + f3	IF				ID				RR	EX1	EX2	WB										(threads 0-7)
w0										RR	EX1	EX2	WB									(threads 8-15)
											RR	EX1	EX2	WB								(threads 16-23)
												RR	EX1	EX2	WB							(threads 24-31)
f4 = f1 + f5					IF				ID				RR	EX1	EX2	WB						(threads 0-7)
w1														RR	EX1	EX2	WB					(threads 8-15)
															RR	EX1	EX2	WB				(threads 16-23)
																RR	EX1	EX2	WB			(threads 24-31)

Register File Organization

- Wide (vector) registers

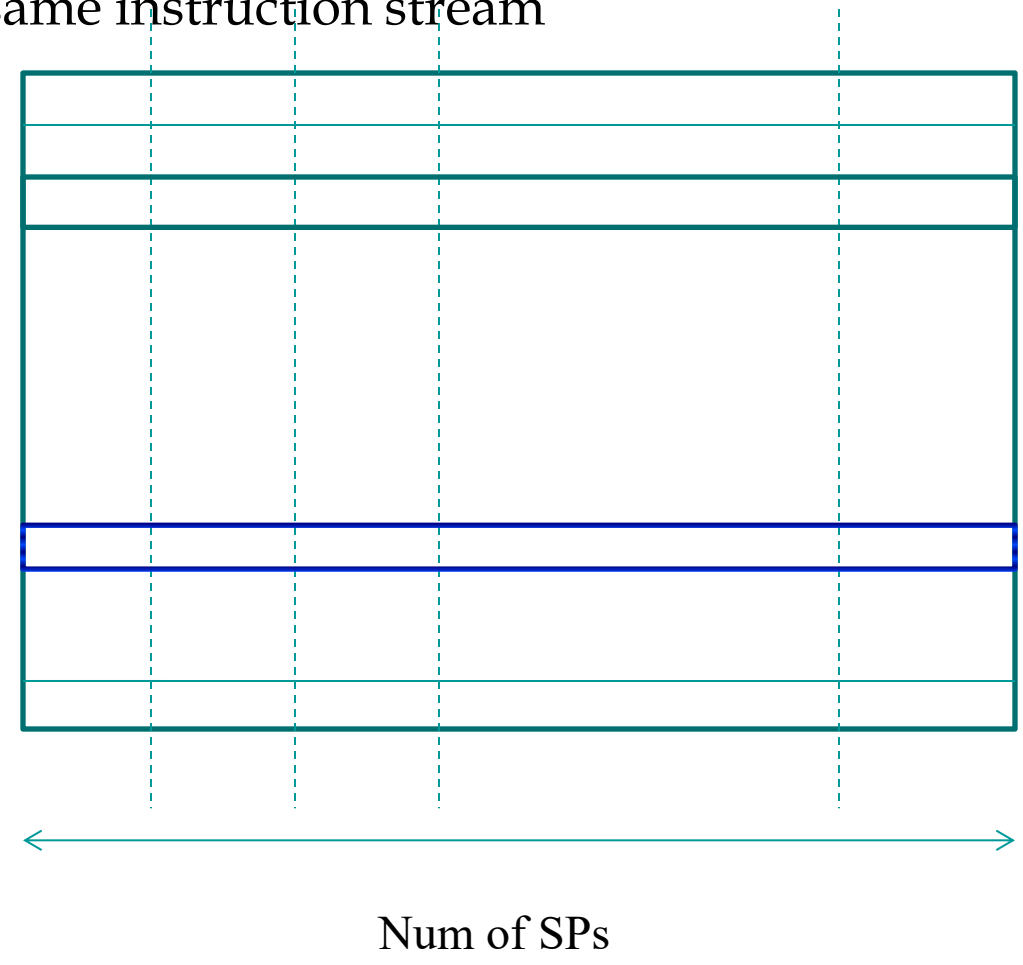




Register File Organization

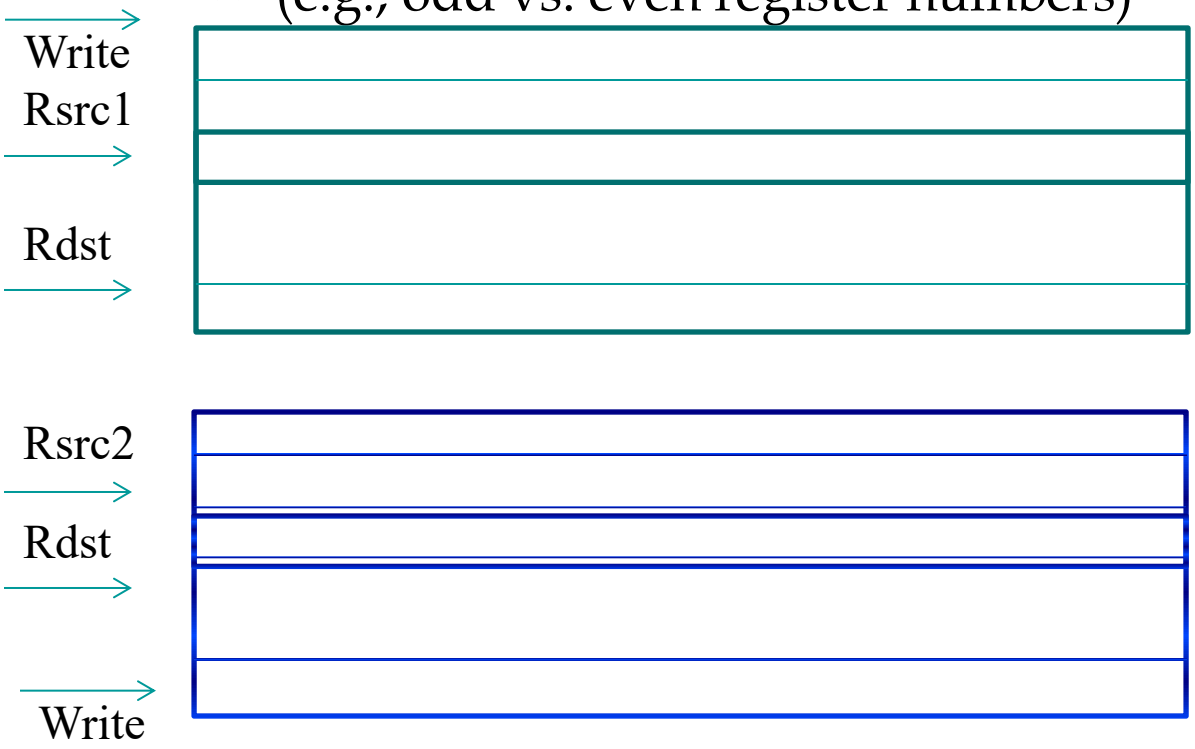
- Register renaming
  - Every thread has the same instruction stream
  - Add r1, r2, r3
  - How to differentiate

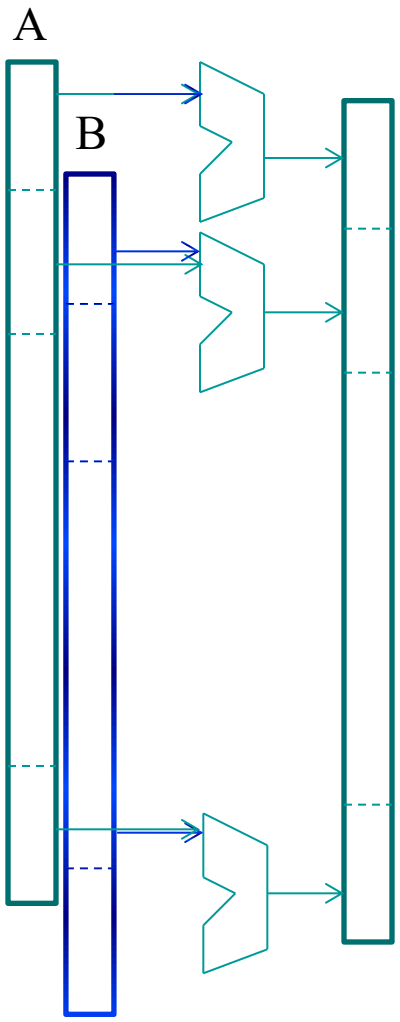
  r1 for different threads?
  - In the same warp?
  - In different warps?



Register File Organization

- Banking
  - (e.g., odd vs. even register numbers)





ILP in GPU pipelines

- Option 1: no more than 1 instruction from a warp can be issued
  - Easy implementation using the ready bit in the warp scheduler
    - Clear the bit when an instruction is issued from the warp
    - Set the bit when the instruction commits during the WB stage
- Option 2: allow more than 1 instruction from a warp to be issued into the pipeline
  - Scoreboard to detect RAW and WAW data hazards
    - One possible design of the scoreboard
      - Ready bit for each physical vector register
  - Ready bit in the warp scheduler is dependent on scoreboard checking
  - Bypassing

Pipeline Execution Diagram (Option 1)

Ex: Two active warps running on an SM with 16 execution pipelines. The add instruction has a 6-cycle latency. The SM uses a round-robin scheduling policy to pick the next ready warp. There is no scoreboard in the SM.

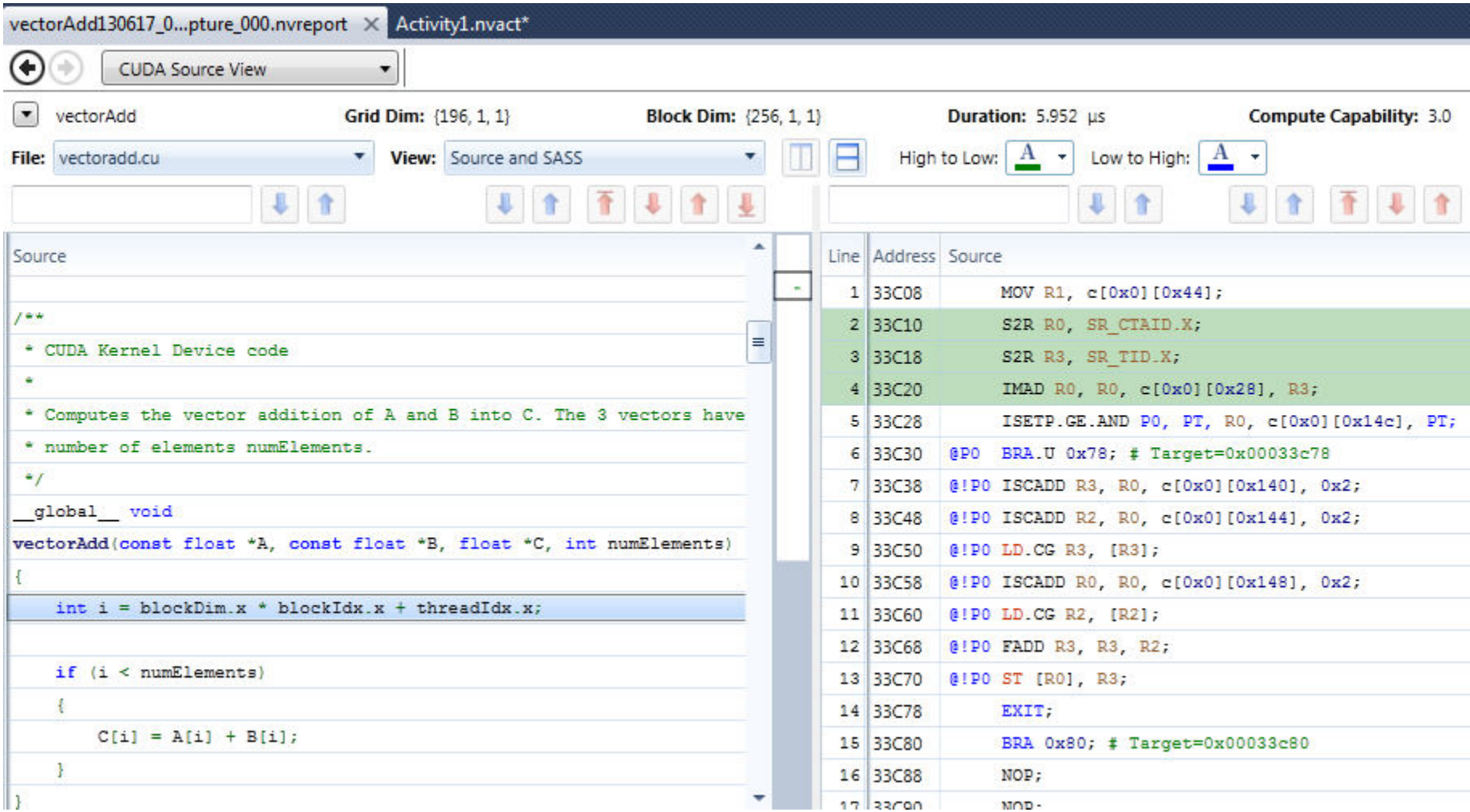
Cycle	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	
f1=f2+f3	IF		ID		RR	EX1	EX2	EX3	EX4	EX5	EX6	WB						threads 0~15
(warp 0)						RR	EX1	EX2	EX3	EX4	EX5	EX6	WB					threads 16~31
f1=f2+f3			IF		ID		RR	EX1	EX2	EX3	EX4	EX5	EX6	WB				threads 0~15
(warp 1)								RR	EX1	EX2	EX3	EX4	EX5	EX6	WB			threads 16~31
f4=f5+f6					IF		ID		s	s	s	s	RR	EX1	EX2	EX3	EX4	threads 0~15
(warp 0)														RR	EX1	EX2	EX3	threads 16~31
F4=f5+f6							IF		ID	s	s	s	s	s	RR	EX1	EX2	threads 0~15
(warp 1)																RR	EX1	threads 16~31

Pipeline Execution (option 2)

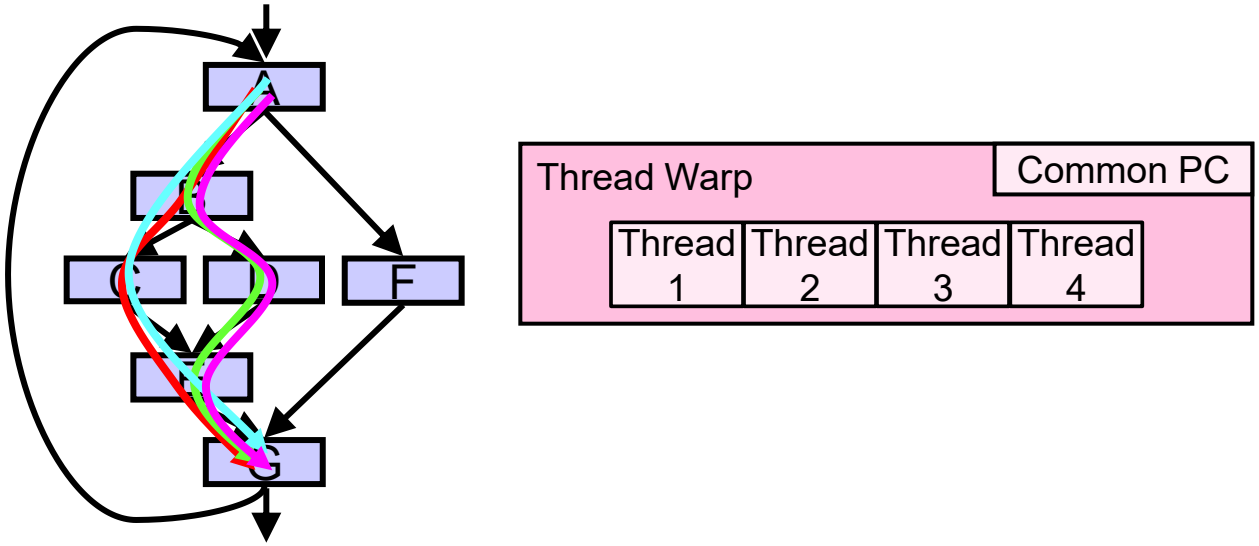
Ex: Two active warps running on an SM with 16 execution pipelines. The add instruction has a 6-cycle latency. The SM uses a round-robin scheduling policy to pick the next ready warp. There is a scoreboard in the SM to support more than 1 insn. from the same warp in the pipeline.

Cycle	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	
f1=f2+f3	IF		ID		RR	EX1	EX2	EX3	EX4	EX5	EX6	WB						threads 0~15
(warp 0)						RR	EX1	EX2	EX3	EX4	EX5	EX6	WB					threads 16~31
f1=f2+f3			IF		ID		RR	EX1	EX2	EX3	EX4	EX5	EX6	WB				threads 0~15
(warp 1)								RR	EX1	EX2	EX3	EX4	EX5	EX6	WB			threads 16~31
f4=f5+f6					IF		ID		RR	EX1	EX2	EX3	EX4	EX5	EX6	WB		threads 0~15
(warp 0)										RR	EX1	EX2	EX3	EX4	EX5	EX6	WB	threads 16~31
F4=f5+f6							IF		ID		RR	EX1	EX2	EX3	EX4	EX5	EX6	threads 0~15
(warp 1)												RR	EX1	EX2	EX3	EX4	EX5	threads 16~31

Example

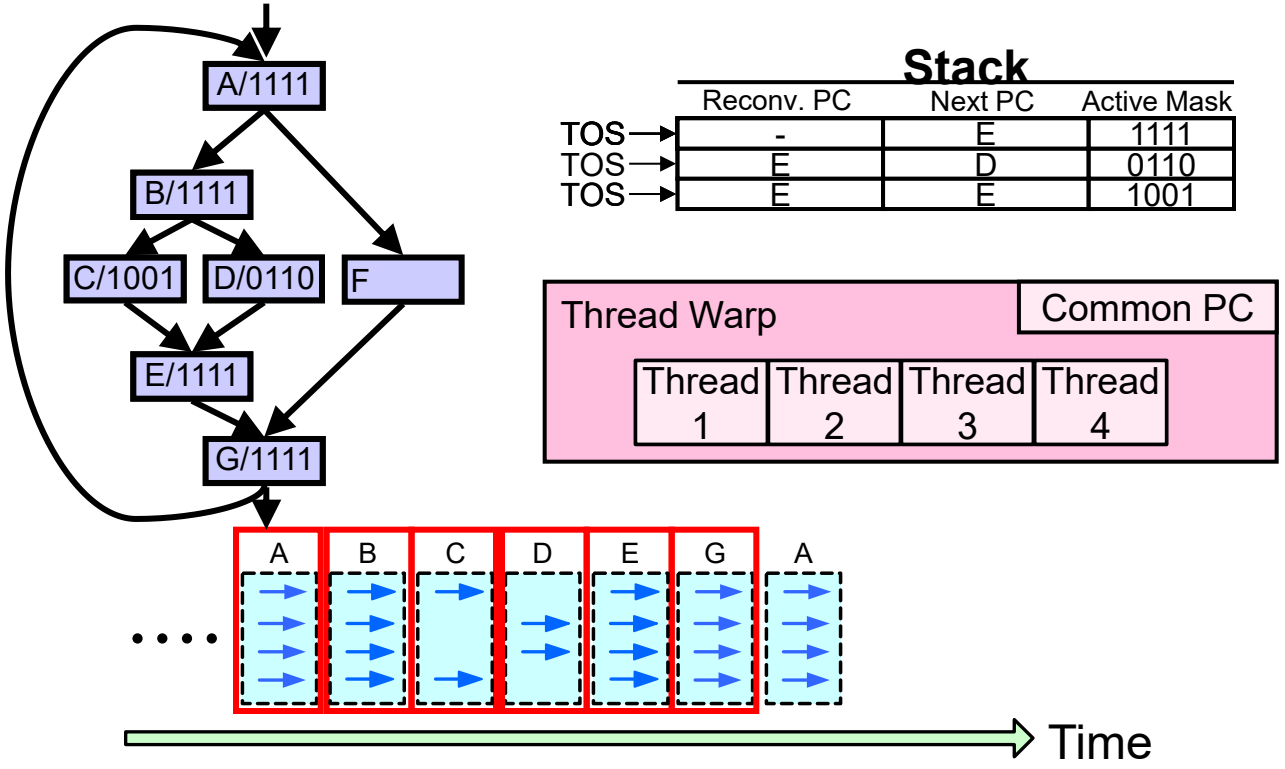

vectorAdd130617_0...pture_000.nvreport
Activity1.nvact*
CUDA Source View
vectorAdd
Grid Dim: {196, 1, 1}
Block Dim: {256, 1, 1}
Duration: 5.952 µs
Compute Capability: 3.0
File: vectoradd.cu
View: Source and SASS
High to Low:
Low to High:
Source
/**
* CUDA Kernel Device code
*
* Computes the vector addition of A and B into C. The 3 vectors have
* number of elements numElements.
*/
__global__ void
vectorAdd(const float *A, const float *B, float *C, int numElements)
{
int i = blockDim.x * blockIdx.x + threadIdx.x;
if (i < numElements)
{
C[i] = A[i] + B[i];
}
}
Line
Address
Source
1 33C08 MOV R1, c[0x0][0x44];
2 33C10 S2R R0, SR_CTAID.X;
3 33C18 S2R R3, SR_TID.X;
4 33C20 IMAD R0, R0, c[0x0][0x28], R3;
5 33C28 ISETP.GE.AND P0, PT, R0, c[0x0][0x14c], PT;
6 33C30 @P0 BRA.U 0x78; # Target=0x00033c78
7 33C38 @!P0 ISCADD R3, R0, c[0x0][0x140], 0x2;
8 33C48 @!P0 ISCADD R2, R0, c[0x0][0x144], 0x2;
9 33C50 @!P0 LD.CG R3, [R3];
10 33C58 @!P0 ISCADD R0, R0, c[0x0][0x148], 0x2;
11 33C60 @!P0 LD.CG R2, [R2];
12 33C68 @!P0 FADD R3, R3, R2;
13 33C70 @!P0 ST [R0], R3;
14 33C78 EXIT;
15 33C80 BRA 0x80; # Target=0x00033c80
16 33C88 NOP;

Branch Divergence/Control Divergence



Source: Wilson W. L. Fung el al. Dynamic Warp Formation and Scheduling for Efficient GPU Control Flow Micro 2007

PDOM Stack Based Reconvergence Model



Source: Wilson W. L. Fung el al. Dynamic Warp Formation and Scheduling for Efficient GPU Control Flow Micro 2007

GPU instruction execution vs. scalar instruction execution

- Ex: Vector add (32 elements)
- Scalar execution in an in-order scalar pipeline

```
     MOV   r5, #0
L:  LOAD   f1, 0(r1)
    LOAD   f2, 0(r2)
    FADD   f3, f1, f2
    STORE  f3, 0(r3)
    ADD    r1, r1, #4
    ADD    r2, r2, #4
    ADD    r3, r3, #4
    ADD    r5, r5, 1
    BRLE   r5, 32, L  /* loop upper bound as 32 */
```

Cycle	1	2	3	4	5	6	7	8	9	10	11
FADD f3,f1,f2	IF	ID	RR	EX1	EX2	EX3	EX4	EX5	WB		
STORE f3, 0(r3)		IF	ID	RR	stall	stall	stall	stall	EX1	EX2	WB
ADD r1,r1, #4			IF	ID	stall	stall	stall	stall	stall	RR	EX

GPU execution

- GPU code

```
/* generate the addresses based on thread id and the base addresses
   of the vectors*/
LOAD   f1, 0(r1)
LOAD   f2, 0(r2)
FADD   f3, f1, f2
STORE f3, 0(r3)
```

- Case 1: one execution pipeline in an SM

Cycle	n	n+1	n+2	n+3	n+4	n+5	n+6	n+7	...	n+37	n+38	
FADD f3,f1,f2	RR	EX1	EX2	EX3	EX4	EX5	WB					Thread 0
		RR	EX1	EX2	EX3	EX4	EX5	WB				Thread 1
			...					...	...			...
				...						WB		Thread 31
STORE f3, 0(r3)								...	...	EX2	WB	Thread 0
									...	EX1	EX2	Thread 1
												...

GPU execution (cont'd)

- GPU code

```
/* generate the addresses based on thread id and the base addresses
   of the vectors*/
LOAD   f1, 0(r1)
LOAD   f2, 0(r2)
FADD   f3, f1, f2
STORE f3, 0(r3)
```

- Case 2: 16 execution pipelines in an SM

Cycle	n	n+1	n+2	n+3	n+4	n+5	n+6	n+7	n+8	n+9	
FADD f3,f1,f2	RR	EX1	EX2	EX3	EX4	EX5	WB				Threads 0~15
		RR	EX1	EX2	EX3	EX4	EX5	WB			Threads 16~31
STORE f3, 0(r3)			stall	stall	stall	stall	stall	RR	EX1	EX2	Threads 0~15
									RR	EX1	Threads 16~31

Supporting Multi-Kernel Execution

